

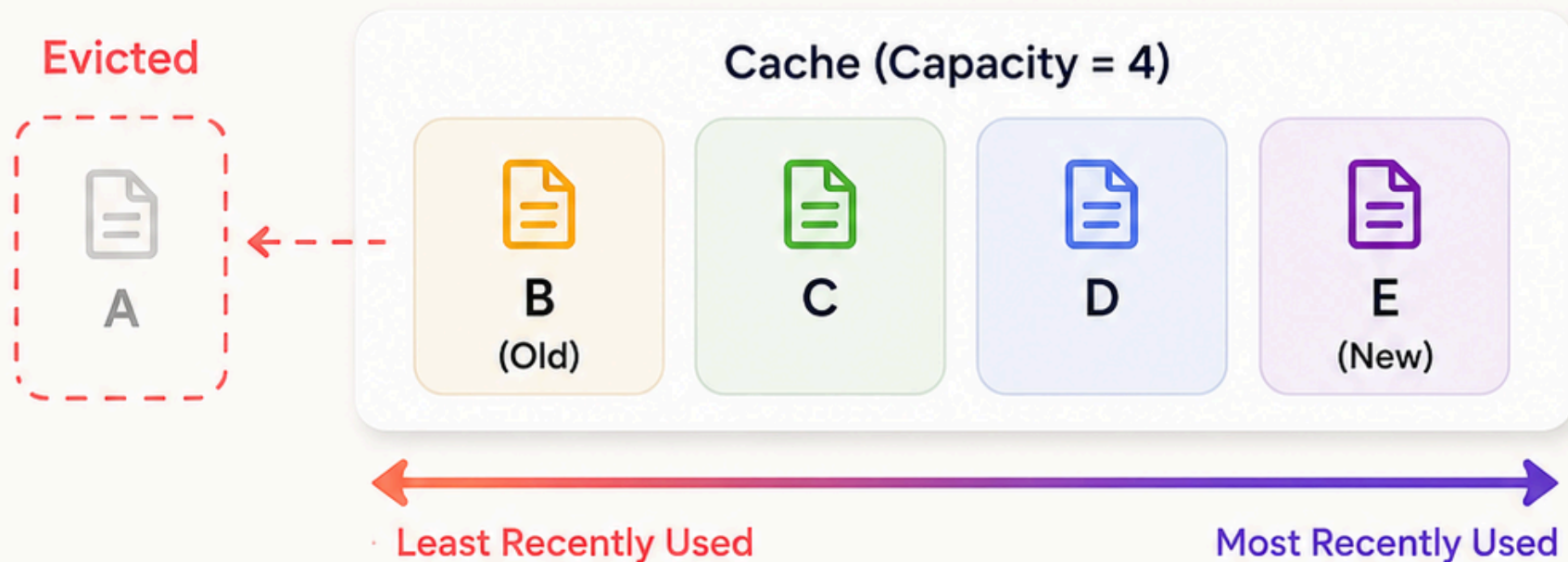
What LRU actually means



Least Recently Used.

A cache eviction rule.

When you hit capacity, you remove whatever **hasn't been touched** in the longest time, **not something random**.



LRU keeps the things you use,
and lets go of the things you don't.

How LRU works



LRU (Least Recently Used) is a cache eviction policy. When the cache is full and we need space, we **remove the item that hasn't been used for the longest time.**

The idea in simple steps

1 Keep items in order of use

The most recently used item is at the end.
The least recently used item is at the front.

2 On access (get)

- If the item exists → move it to the end.
- If not → it's a miss.

3 On insert/update (put)

- If key exists → update value and move to end.
- If key doesn't exist → add to end.
- If size > capacity → remove the first item (LRU).

Pseudo code

```
on GET(key):  
    if key not in cache:  
        return NOT_FOUND  
    move key to the end (mark as most recent)  
    return cache[key]  
  
on PUT(key, value):  
    if key in cache:  
        update its value  
        move key to the end  
    else:  
        add key at the end  
  
if size > capacity:  
    remove the first item (least recent)
```



Always keep recency order.
Use = move to end. Overflow = remove from front.

Step one, initialize



A **capacity** number and a **Map**.

JavaScript's Map **preserves insertion order**, and that one property carries most of the solution.

```
class LRUCache {
  constructor(capacity) {
    if (capacity <= 0) {
      throw new Error('Capacity must be positive');
    }
    this.capacity = capacity; // Max items
    this.cache = new Map();   // Stores key-value pairs
  }
}

// Usage
const lru = new LRUCache(3);
```

What we have

capacity



3

cache



Map(0) { }



Map **remembers the order** in which keys are added. That's our secret weapon.



We're ready with a capacity limit and an ordered store. Next step: **add** and **move**.

Step two, the get method



Direct lookup by key, **O(1)**, no traversal involved.
Then **delete** and **reinsert** that key to mark it as **most recently used**.

```
class LRUCache {
  constructor(capacity) {
    if (capacity <= 0) throw new Error('Capacity must be positive');
    this.capacity = capacity;
    this.cache = new Map(); // key -> value
  }

  get(key) {
    if (!this.cache.has(key)) return -1; // Not found

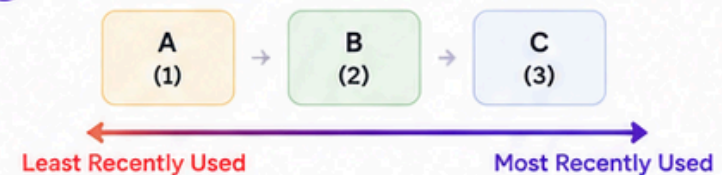
    const value = this.cache.get(key); // O(1) lookup
    this.cache.delete(key);           // Remove existing
    this.cache.set(key, value);        // Re-insert as MRU
    return value;
  }
}

// Usage
const lru = new LRUCache(3);
lru.cache.set('A', 1);
lru.cache.set('B', 2);
lru.cache.set('C', 3);
console.log(lru.get('B')); // 2

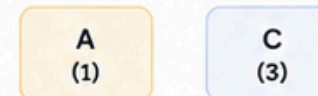
// Cache order is now: A -> C -> B (B is most recently used)
```

How get('B') works

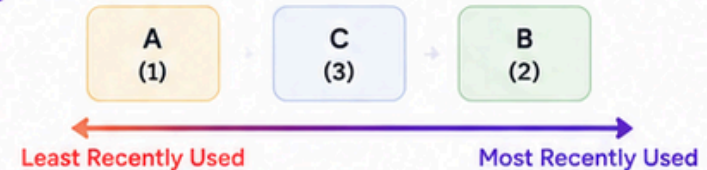
1 Cache before get('B')



2 Delete 'B'



3 Re-insert 'B' (MRU)



Map keeps insertion order.
By moving the key to the end,
we mark it as **most recently used**.



get() is fast, simple, and keeps our usage order correct.
Next: handle capacity with the **put** method.

Step three, the put method



If the key exists, **delete it first**. Set the new value.
If size now exceeds capacity, evict the oldest key
using `cache.keys().next().value`.

```
class LRUCache {
  constructor(capacity) {
    if (capacity <= 0) {
      throw new Error('Capacity must be positive');
    }
    this.capacity = capacity;
    this.cache = new Map(); // key -> value
  }

  put(key, value) {
    // 1. If key exists, remove it first
    if (this.cache.has(key)) {
      this.cache.delete(key);
    }

    // 2. Set new value (moves it to the end)
    this.cache.set(key, value);

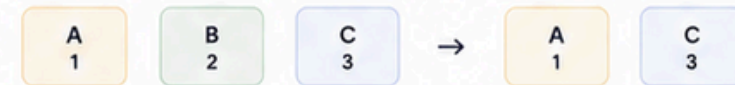
    // 3. If over capacity, evict least recently used (first key)
    if (this.cache.size > this.capacity) {
      const lruKey = this.cache.keys().next().value; // oldest key
      this.cache.delete(lruKey); // evict it
    }
  }
}

// Usage
const lru = new LRUCache(3);
lru.put('A', 1);
lru.put('B', 2);
lru.put('C', 3);
lru.put('D', 4); // Evicts 'A' (the oldest)
console.log([...lru.cache]); // [ ['B', 2], ['C', 3], ['D', 4] ]
```

How put() works

- 1 Key exists? Delete it first.

Before `put('B', 20)`



Least Recently Used

Most Recently Used

- 2 Set the new value (goes to the end)

After `set('B', 20)`



Least Recently Used

Most Recently Used

- 3 If size > capacity, evict the oldest key

Capacity = 3

`put('B', 4)`



Least Recently Used

Most Recently Used



`cache.keys().next().value` gives the **first (oldest)** key in insertion order.



`put()` keeps the cache within capacity
and always **maintains the correct usage order**.